RAMAIAH
Institute of Technology

Department of Computer Science and Engineering

Other CIE Component – 10 marks

SRS Documentation

Intelligent Self-Healing Kubernetes Platform using AI-Driven Observability and Automated Remediation

Submitted for the course Project Work-1(Phase-1) – MCSP34

by

NANDAN NITYANAND BHATKAL **1MS25SCS19**

Under the guidance of

Prof. Darshana A Naik
Assistant Professor

M S RAMAIAH INSTITUTE OF TECHNOLOGY

(Autonomous Institute, Affiliated to VTU)
BANGALORE-560054
www.msrit.edu
September 2026

Revision History

Version	Date	Description	Author
0.1	Week 1	Initial draft: problem statement, objectives, tech stack	Nandan
0.5	Week 2	Literature survey and research-gap analysis incorporated	Nandan
1.0	Week 3	First complete SRS: full functional and non-functional requirements	Nandan

Contents

Revision History **1**

1 Introduction **4**
1.1 Purpose . . . 4
1.2 Document Conventions . . . 4
1.3 Intended Audience and Reading Suggestions . . . 4
1.4 Product Scope . . . 4
1.5 References . . . 5

2 Overall Description **5**
2.1 Product Perspective . . . 5
2.2 Product Functions . . . 5
2.3 User Classes and Characteristics . . . 6
2.4 Operating Environment . . . 6
2.5 Design and Implementation Constraints . . . 6
2.6 User Documentation . . . 6
2.7 Assumptions and Dependencies . . . 7

3 External Interface Requirements **7**
3.1 User Interfaces . . . 7
3.2 Hardware Interfaces . . . 7
3.3 Software Interfaces . . . 7
3.4 Communication Interfaces . . . 7

4 System Features **7**
4.1 Multi-Signal Observability Collection . . . 7
4.2 Real-Time Anomaly Detection . . . 8
4.3 Predictive Failure Analysis . . . 8
4.4 Policy-Driven Automated Remediation . . . 9
4.5 Fault-Injection and Evaluation Testbed . . . 9
4.6 Comparative Evaluation and Reporting . . . 9

5 Other Nonfunctional Requirements **10**
5.1 Performance Requirements . . . 10
5.2 Safety Requirements . . . 10
5.3 Security Requirements . . . 10
5.4 Software Quality Attributes . . . 10
5.5 Business Rules . . . 10

6 Other Requirements **11**

A Glossary **12**

B Analysis Model: System Architecture **12**

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of the **Intelligent Self-Healing Kubernetes Platform**, an AI-driven observability and automated-remediation system for Kubernetes clusters. It is written to guide the design (HLD/LLD) and implementation carried out across Semester 3 (Project-I) and Semester 4 (Project-II), and to give the project guide and review panel a single, unambiguous reference for what the system is expected to do.

1.2 Document Conventions

Requirements are identified with the prefixes **FR** (Functional Requirement) and **NFR** (Non-Functional Requirement), numbered by feature area, e.g. FR-2.3. Each requirement uses **shall** to denote a mandatory capability, following IEEE/RFC-2119 convention. Priority is marked as **High**, **Medium**, or **Low**, reflecting whether the requirement is targeted for Semester 3 or Semester 4.

1.3 Intended Audience and Reading Suggestions

This document is intended for:

- The project guide and review panel, to evaluate scope and feasibility at each review.
- The developer, as the implementation blueprint for both semesters.
- Future readers of the project report, to understand system boundaries without needing the full design documents.

Readers evaluating scope should focus on Sections 2 and 4; readers evaluating engineering rigor should focus on Sections 4 and 5.

1.4 Product Scope

Kubernetes deployments experience recurring failure modes (pod crashes, memory leaks, CPU spikes, network bottlenecks, service outages) that current monitoring stacks detect and alert on, but do not resolve autonomously. Kubernetes' own self-healing primitives (liveness/readiness probes, ReplicaSet controllers, Horizontal Pod Autoscaler) recover only a narrow, predefined set of failures, do not correlate signals across metrics, logs, and traces, and are not guaranteed to recover the system within an acceptable time window.

The product is a closed-loop platform that (a) continuously observes a Kubernetes cluster across metrics, logs, and traces; (b) applies a lightweight, classical machine-learning model to detect anomalies and flag early-warning failure trends; and (c) triggers a policy-driven remediation action – pod restart, deployment scaling, release rollback, or human escalation – without requiring an LLM in the remediation decision path. The system is evaluated empirically, using controlled fault injection, against standard SRE metrics: Availability, Mean Time to Recovery (MTTR), Mean Time Between Failures (MTBF), and Detection Accuracy.

This positions the platform deliberately as a lightweight, reproducible, classical-ML alternative to the LLM-agent-based remediation systems that have appeared in the literature since 2024 – an explicit scope decision, not an oversight, and one this SRS treats as a constraint (see Section 2.5).

1.5 References

[1] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA, USA: O'Reilly Media, 2016.
[2] Kubernetes Documentation, "Self-Healing." [Online]. Available: `https://kubernetes.io/docs/concepts/architecture/self-healing`
[3] Y. Matsuo and D. Ikegami, "Performance Analysis of Anomaly Detection Methods for Application System on Kubernetes with Auto-scaling and Self-healing," in *Proc. 2021 17th Int. Conf. Netw. Service Manage. (CNSM)*, 2021.
[4] Z. Yazdanparast, "A Survey on Self-healing Software System," *arXiv:2403.00455*, 2024.
[5] X. Zou *et al.*, "Towards Generalizable Context-aware Anomaly Detection: A Large-scale Benchmark in Cloud Environments," *arXiv:2508.01844*, 2025.
[6] K8sGPT Project (CNCF Sandbox). [Online]. Available: `https://github.com/k8sgpt-ai/k8sgpt`
[7] C. Wang, T. Yuan, C. Hua, L. Chang, X. Yang, and Z. Qiu, "Integrating Large Language Models with Cloud-Native Observability for Automated Root Cause Analysis and Remediation," in *Proc. 2025 3rd Int. Conf. Artif. Intell., Syst. Netw. Security (AISNS)*, 2025.
[8] M. S. Ardebili and A. Bartolini, "KubeIntellect: A Modular LLM-Orchestrated Agent Framework for End-to-End Kubernetes Management," *arXiv:2509.02449*, 2025.
[9] Y. Chen *et al.*, "STRATUS: A Multi-Agent System for Autonomous Reliability Engineering of Modern Clouds," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
[10] Dynatrace, "Dynatrace Advances AIOps with Preventive Operations," Feb. 2025.
[11] Chaos Mesh (CNCF). [Online]. Available: `https://chaos-mesh.org`
[12] LitmusChaos (CNCF). [Online]. Available: `https://litmuschaos.io`
[13] AIOpsLab (Microsoft Research). [Online]. Available: `https://github.com/microsoft/AIOpsLab`

2 Overall Description

2.1 Product Perspective

The platform is a new, self-contained system that sits alongside a Kubernetes cluster rather than modifying Kubernetes core components. It consumes telemetry through standard, vendor-neutral interfaces (Prometheus exposition format, Loki push API, OpenTelemetry/Jaeger trace ingestion) and acts on the cluster exclusively through the standard Kubernetes API, so it can be deployed onto any conformant cluster – local (Minikube/Kind) or managed cloud (EKS/GKE/AKS) – without modification.

2.2 Product Functions

At a high level, the platform:

- Collects and correlates metrics, logs, and traces from a monitored application running on Kubernetes.
- Detects anomalous system behaviour in real time using a classical ML model.
- Flags early-warning failure trends before they cause service degradation.
- Selects and executes a remediation action according to a configurable policy.
- Verifies remediation outcome and rolls back / escalates on failure.
- Injects controlled faults and reports Availability, MTTR, MTBF, and Detection Accuracy.

2.3 User Classes and Characteristics

- **Cluster / SRE Operator** – the primary user in a deployed setting; consumes Grafana dashboards and remediation/audit logs, and configures policies and thresholds. Assumed to have working Kubernetes knowledge.
- **Project Guide / Review Panel** – evaluates the system against its stated objectives and metrics; does not operate the system directly.
- **Developer** – designs, implements, trains models, and evaluates the platform across both semesters.

2.4 Operating Environment

	Minimum	**Recommended**
CPU	4 cores	8 cores
RAM	8 GB	16 GB or more
Storage	40 GB free (SSD preferred)	100 GB free SSD
Operating System	Ubuntu 20.04+ / WSL2 / macOS 12+	Ubuntu 24.04 LTS

Software platform: Docker Engine; Kubernetes tooling (`kubectl`, Minikube or Kind, Helm); Prometheus, Loki, Jaeger (via an OpenTelemetry Collector) and Grafana; Python 3.10+ with Scikit-learn, Pandas, and NumPy for the detection/prediction models; a Kubernetes client library (Go `client-go` or Python `kubernetes`) for the remediation controller; Chaos Mesh or LitmusChaos for fault injection; Git for version control.

2.5 Design and Implementation Constraints

- **No LLM in the remediation decision path.** Detection and remediation-policy selection use classical ML and rule-based policies only, to keep inference sub-second and avoid a dependency on external LLM APIs or GPU infrastructure – a deliberate positioning against the LLM-agent systems surveyed in the literature.
- **Tiered tool stack.** Kubernetes, Prometheus, Grafana, Loki, Scikit-learn, and the remediation controller are the core, load-bearing components. Terraform, Ansible, Jenkins, Argo CD, and Istio are supporting/stretch components and are not required for the core research contribution.
- **Two-semester timeline.** Semester 3 (Project-I) delivers architecture, environment setup, the observability framework, and the fault-injection testbed. Semester 4 (Project-II) delivers the ML detection/prediction models, the remediation engine, and the comparative evaluation.
- **Solo development.** The system is designed and built by a single developer; scope is deliberately bounded to what is feasible within that constraint (see the tiered stack above).
- **Open-source only.** Every tool in the stack is open source, to keep the platform reproducible without licensing cost.

2.6 User Documentation

The project will deliver a README with setup instructions, a configuration reference for detection thresholds and remediation policies, and the final project report (covering SRS, HLD/LLD, and evaluation results).

2.7 Assumptions and Dependencies

- A Kubernetes cluster (local or cloud) is available and reachable via `kubectl` for both development and evaluation.
- The monitored application is instrumented, or can be instrumented, to expose Prometheus-compatible metrics.
- Fault injection via Chaos Mesh/LitmusChaos is permitted on the evaluation cluster (i.e., evaluation does not run against a shared production cluster).
- Sufficient historical telemetry (normal-operation data) can be collected to train the anomaly-detection model before evaluation begins.

3 External Interface Requirements

3.1 User Interfaces

Grafana dashboards present live metrics, active anomalies, and a remediation-action timeline. A command-line interface exposes policy configuration and manual override (pause/resume automated remediation).

3.2 Hardware Interfaces

None directly; the platform interacts only with the virtualized compute, storage, and network resources exposed by the Kubernetes cluster's nodes.

3.3 Software Interfaces

- **Kubernetes API server** – read cluster/pod/deployment state; execute restart, scale, and rollback actions.
- **Prometheus HTTP API** – query time-series metrics for feature extraction.
- **Loki API** – query and stream log data for correlation.
- **Jaeger / OpenTelemetry Collector** – ingest and query distributed traces.
- **Chaos Mesh / LitmusChaos API** – programmatically trigger fault-injection experiments during evaluation.

3.4 Communication Interfaces

All internal communication uses HTTP/REST or gRPC (for the Kubernetes API). Remediation and escalation alerts are delivered over a webhook-based channel (e.g., Slack or email webhook).

4 System Features

4.1 Multi-Signal Observability Collection

Description: Continuous collection and correlation of metrics, logs, and traces from the monitored cluster. *Priority:* High (Semester 3).

ID	Requirement	Priority
FR-1.1	The system shall collect CPU, memory, network, and disk-I/O metrics from all pods and nodes via Prometheus at a configurable scrape interval (default 15s).	High
FR-1.2	The system shall aggregate container and application logs via Loki.	High
FR-1.3	The system shall capture distributed traces of inter-service requests via Jaeger.	Medium
FR-1.4	The system shall route metrics, logs, and traces through a single OpenTelemetry Collector pipeline.	Medium
FR-1.5	The system shall visualize collected telemetry on Grafana dashboards.	High
FR-1.6	The system shall retain telemetry for a configurable period (default 7 days) to support model training and historical analysis.	Medium

4.2 Real-Time Anomaly Detection

Description: Detection of abnormal system behaviour from live metrics, correlated with logs. *Priority:* High (Semester 3/4 boundary).

ID	Requirement	Priority
FR-2.1	The system shall extract a feature vector from Prometheus metrics at each evaluation interval.	High
FR-2.2	The system shall compute an anomaly score for each evaluation window using a trained classical ML model (e.g., Isolation Forest or One-Class SVM).	High
FR-2.3	The system shall classify a window as anomalous when its anomaly score exceeds a configurable threshold.	High
FR-2.4	The system shall correlate a detected metric anomaly with concurrent log entries before confirming it, to reduce false positives.	Medium
FR-2.5	The system shall log every detection decision (score, threshold, timestamp, correlated evidence) for auditability.	High

4.3 Predictive Failure Analysis

Description: Early-warning forecasting of failure trends before they cause degradation. *Priority:* High (Semester 4).

ID	Requirement	Priority
FR-3.1	The system shall model short-horizon temporal trends in key resource and performance metrics.	High
FR-3.2	The system shall raise an early-warning alert when a metric trend is projected to cross a critical threshold within a configurable lookahead window (default 5 minutes).	High
FR-3.3	The system shall label its output as either an early-warning (predictive) alert or a confirmed-anomaly (reactive) alert.	Medium

4.4 Policy-Driven Automated Remediation

Description: Executing a bounded, verifiable remediation action in response to a detection or early-warning signal. *Priority:* High (Semester 4).

ID	Requirement	Priority
FR-4.1	The system shall maintain a policy mapping anomaly/failure type to a remediation action (restart pod, scale deployment, rollback release, or alert only).	High
FR-4.2	On a confirmed anomaly or early-warning trigger, the system shall select and execute the mapped remediation action via the Kubernetes API.	High
FR-4.3	The system shall re-evaluate target metrics within a configurable observation window (default 60s) after executing an action, to verify its outcome.	High
FR-4.4	If the observed outcome does not improve within the observation window, the system shall roll back the executed action where applicable and escalate to a human operator.	High
FR-4.5	The system shall enforce a rate limit on automated remediation actions per resource per time window, to prevent remediation loops.	Medium
FR-4.6	The system shall log every remediation action, its trigger, and its verified outcome.	High

4.5 Fault-Injection and Evaluation Testbed

Description: Controlled, repeatable fault injection to support empirical evaluation. *Priority:* High (Semester 3).

ID	Requirement	Priority
FR-5.1	The system shall support controlled injection of pod-crash, memory-leak, CPU-spike, and network-bottleneck faults via Chaos Mesh or LitmusChaos.	High
FR-5.2	The system shall record the injection timestamp, fault type, and affected resource for every experiment.	High
FR-5.3	The system shall compute Detection Latency (fault injection to detection) and Recovery Time (detection to verified remediation) for every experiment.	High

4.6 Comparative Evaluation and Reporting

Description: Quantifying platform effectiveness against a Kubernetes-native baseline. *Priority:* High (Semester 4).

ID	Requirement	Priority
FR-6.1	The system shall compute Availability, MTTR, MTBF, and Detection Accuracy over a defined evaluation period.	High
FR-6.2	The system shall report the platform's own resource utilization (CPU/memory/network) during evaluation.	Medium

ID	Requirement	Priority
FR-6.3	The system shall support a baseline mode (Kubernetes-native self-healing only, ML/remediation layer disabled) to enable direct comparison.	High

5 Other Nonfunctional Requirements

5.1 Performance Requirements

NFR-1 Anomaly-detection inference shall complete in under 1 second per evaluation window (classical ML, no LLM inference in the loop).

NFR-2 The platform's own monitoring and control-loop components shall not add more than approximately 10% CPU and 5% memory overhead to the monitored cluster's baseline usage.

5.2 Safety Requirements

NFR-3 No remediation action shall be executed without first passing the configured policy and rate-limit checks (FR-4.1, FR-4.5).

NFR-4 Every remediation action shall be either reversible by the system (FR-4.4) or logged with sufficient detail for manual reversal.

5.3 Security Requirements

NFR-5 The remediation controller shall operate under a least-privilege Kubernetes RBAC service account, scoped only to the actions in FR-4.1.

NFR-6 All remediation actions shall be audit-logged with actor, action, target resource, and timestamp.

5.4 Software Quality Attributes

NFR-7 **Reliability:** a failure of the platform's own components shall not prevent Kubernetes-native self-healing (probes, ReplicaSet controllers) from continuing to function.

NFR-8 **Portability:** the platform shall deploy on any conformant Kubernetes distribution (Minikube, Kind, or managed cloud Kubernetes) through configuration only, with no code changes.

NFR-9 **Maintainability:** detection thresholds and remediation policies shall be externally configurable without redeploying the platform.

NFR-10 **Scalability:** the observability pipeline shall sustain its default scrape interval for at least 50 monitored pods without noticeable degradation, matching the scale of the evaluation testbed.

5.5 Business Rules

NFR-11 Evaluation results reported in the project report shall be derived from real, controlled fault injection on a running cluster, not from simulated or synthetic logs alone.

6 Other Requirements

All third-party tools used (Kubernetes, Prometheus, Grafana, Loki, Jaeger, Chaos Mesh, LitmusChaos, Scikit-learn) are open source under licenses (Apache 2.0 / BSD / MIT) compatible with academic use and redistribution in the project report.

A Glossary

Term	Definition
MTTR	Mean Time to Recovery – average time from failure onset to verified recovery.
MTBF	Mean Time Between Failures – average operational time between successive failures.
Availability	Percentage of total time the monitored system is in a healthy, serving state.
HPA	Horizontal Pod Autoscaler – Kubernetes' built-in scaling controller.
CRD	Custom Resource Definition – mechanism for extending the Kubernetes API.
Operator	A Kubernetes controller that encodes operational knowledge to manage an application.
Anomaly Score	A numeric output of the detection model indicating how unusual an observation is.
Chaos Engineering	The discipline of injecting controlled failures to test system resilience.

B Analysis Model: System Architecture

Figure 1 shows the end-to-end data and control flow: telemetry flows up from the monitored application through the observability layer into the detection engine, and remediation decisions flow back down into the cluster, closing the loop.

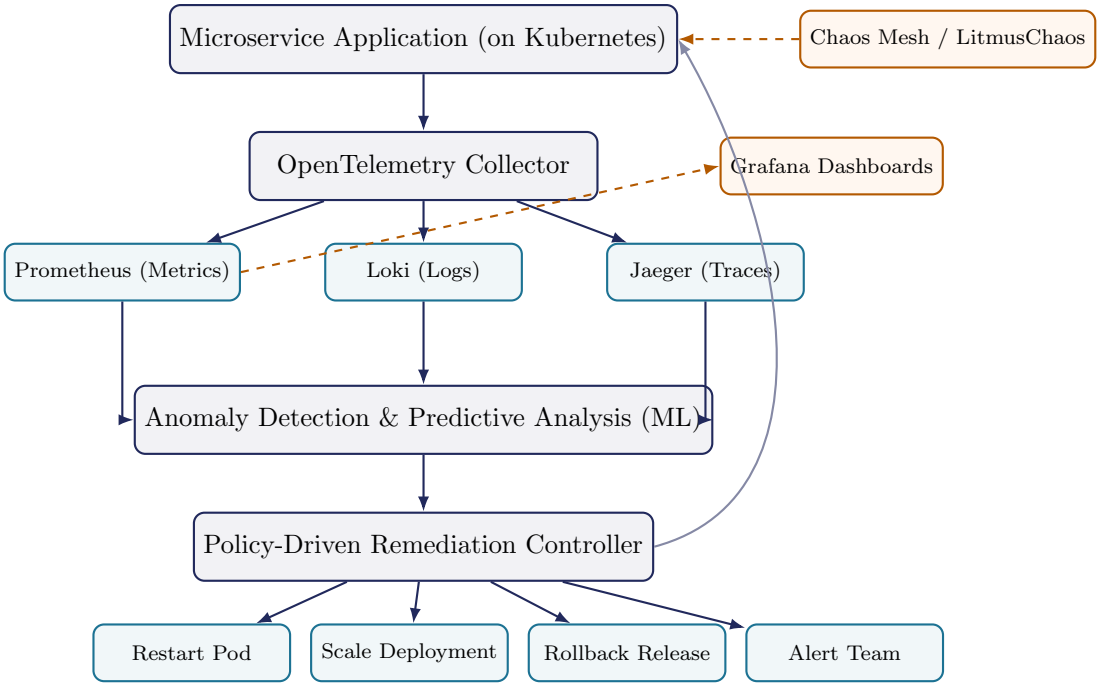


Figure 1: High-level system architecture and data/control flow. Solid arrows: telemetry and remediation flow. Dashed arrows: fault injection and visualization. The curved arrow shows the closed-loop feedback from remediation back into the cluster.